Manual routing & serving JSON

10 min read · Updated 24 Jul 2026 · Topic 32 of 70

Bookmark · Mark done

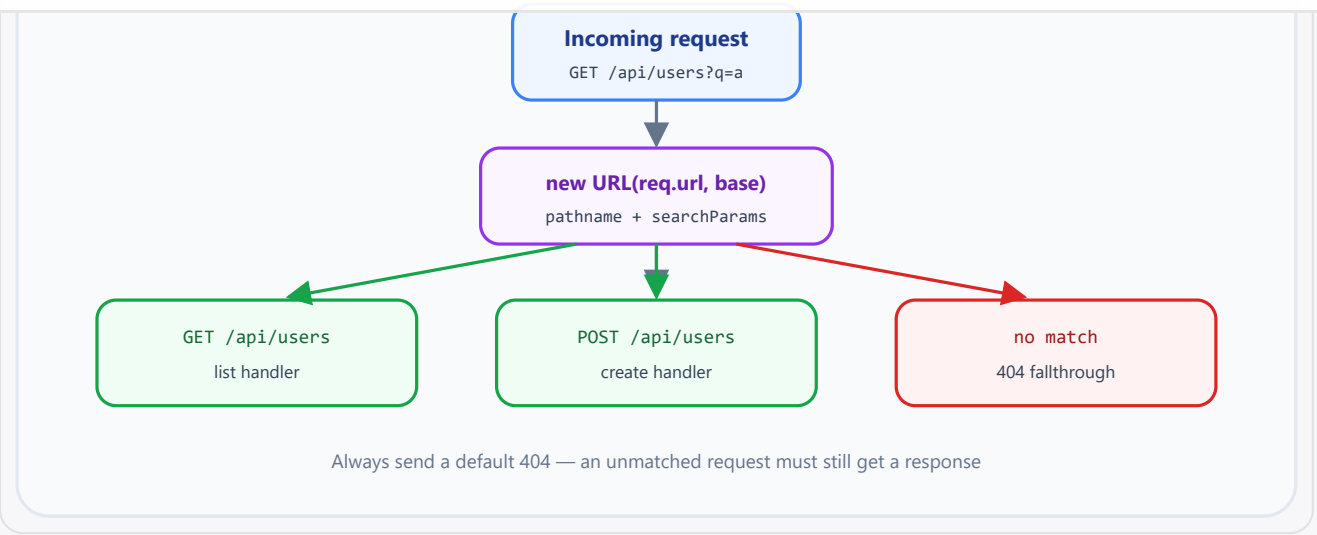
Why it matters

A real API answers *different requests differently*: `GET /users` lists, `POST /users` creates, `GET /users/42` fetches one, and anything else returns 404. That dispatch — deciding which handler runs based on method and path — is **routing**. Frameworks give you `app.get('/users', ...)`, but under the hood they are doing exactly what you will build by hand here: reading `req.method` and the URL pathname and choosing a function.

Writing routing yourself once is worth a dozen framework tutorials. It shows you where path parameters come from, why route order matters, how query strings are parsed, and why a missing default case leaves requests hanging. It is also a staple interview exercise — "implement a tiny router" — because it exercises string handling, HTTP semantics, and clean control flow all at once.

The concept, simply

Routing is a two-key lookup: **method + path → handler**. You take the incoming `req.method` and the URL's `pathname`, compare them against the routes you know about, and call the matching handler. If nothing matches, you send a `404`. If the path matches but the method does not, you send a `405`. Everything else — parsing the query string, extracting an id from `/users/42`, serving JSON — is detail layered on top of that core decision.



Manual routing matches method plus path; an unmatched request must still get a default 404.

How it works

Separating path from query. `req.url` bundles both (`/users?active=1`). Build a `URL` object — `const u = new URL(req.url, \`http://${req.headers.host}\`)` — and route on `u.pathname` (`/users`), reading filters from `u.searchParams`. Never route on the raw `req.url`, or a query string would break every equality check.

Static vs dynamic paths. A static route is a literal string compare: `pathname === '/users'`. A dynamic route has a parameter, like `/users/:id`. Two common techniques: split the pathname on `/` and inspect segments (`parts[1] === 'users' && parts[2]`), or match a `RegExp` with a capture group (`/^\/users\/([^/]+)$/`) and read the captured id. Real routers precompile a set of such patterns.

Method matters. The same path serves different verbs. Structure the dispatch so that a matched path checks the method and returns `405` (with an `Allow` header) when the verb is unsupported, rather than falling through to `404` — that distinction is semantically correct and helps clients.

Query parsing. `u.searchParams` is a `URLSearchParams` instance: `.get('q')` for a single value, `.getAll('tag')` for repeats, `Object.fromEntries(u.searchParams)` for a plain object (note: that collapses repeated keys to the last). Values are already percent-decoded.

removes repetition and guarantees every response has the right header and a single `end()`. The key discipline: *every branch must terminate the response exactly once*, and there must be a catch-all 404 so no request escapes unanswered.

Order and fallthrough. Routes are evaluated in order; the first match wins. Put specific routes before general ones. After all checks, an explicit `else` sends the 404 — this is the single most common bug when hand-rolling routing: forgetting the default leaves unmatched requests to hang until the client times out.

Normalising the path before you match. Two URLs that look the same to a human can be different strings to a naive router. `/Users` and `/users` differ in case; `/users/` has a trailing slash; `/users/%34%32` is a percent-encoded `/users/42`. The `URL` class already percent-decodes `pathname` for you, but it does *not* lower-case or strip trailing slashes — those are policy decisions you make. A common approach is to canonicalise once at the top of the handler: lower-case the pathname if your routes are case-insensitive, and strip a single trailing slash (except for the root `/`) so `/users` and `/users/` hit the same handler. Do this *before* any comparison so every route benefits, and be deliberate about it — some APIs treat a trailing slash as a distinct "collection" resource on purpose.

Matching strategies at a glance. The three techniques you will reach for scale differently:

Strategy	Good for	Cost
`if`/`else` ladder	A handful of routes; learning	O(n) string compares; unwieldy past ~10 route
Object/`Map` lookup by `"METHOD path"`	Many *static* routes	O(1) lookup, but cannot express `:param` seg

Compiled `RegExp` list	Dynamic routes with params	O(n) regex tests; what Express/Fastify do und

Most real frameworks combine these: a fast structure (hash map or radix/trie tree) keyed on static segments, with regex only for the dynamic pieces. Understanding the trade-off is exactly what “how would this scale to 500 routes?” is probing for in an interview.

Code examples

A compact JSON API with static and dynamic routes, method handling, and a 404 fallback:

JAVASCRIPT Copy

```
const http = require('node:http');

const users = new Map([[1, { id: 1, name: 'Ada' }]]);
let nextId = 2;

function sendJson(res, status, data) {
  const body = JSON.stringify(data);
  res.writeHead(status, {
    'Content-Type': 'application/json; charset=utf-8',
    'Content-Length': Buffer.byteLength(body),
  });
  res.end(body);
}

async function readJson(req) {
  const chunks = [];
  for await (const c of req) chunks.push(c);
  return chunks.length ? JSON.parse(Buffer.concat(chunks).toString('utf8')
}

const server = http.createServer(async (req, res) => {
  const url = new URL(req.url, `http://${req.headers.host}`);
  const path = url.pathname;
```

```
    return sendJson(res, 200, [...users.values()].slice(0, limit));
  }

  // POST /users → create
  if (path === '/users' && req.method === 'POST') {
    const body = await readJson(req);
    if (!body.name) return sendJson(res, 400, { error: 'name required' });
    const user = { id: nextId++, name: body.name };
    users.set(user.id, user);
    return sendJson(res, 201, user);
  }

  // GET /users/:id → fetch one
  const match = path.match(/^\/users\/(\d+)$/);
  if (match && req.method === 'GET') {
    const user = users.get(Number(match[1]));
    return user
      ? sendJson(res, 200, user)
      : sendJson(res, 404, { error: 'not found' });
  }

  // Path matched but wrong method → 405
  if (path === '/users') {
    res.writeHead(405, { Allow: 'GET, POST' }).end();
    return;
  }

  // Nothing matched → 404
  sendJson(res, 404, { error: 'route not found' });
});

server.listen(3000);
```

A slightly more scalable pattern — a route table keyed by `METHOD path`, which is closer to how minimal routers work:

JAVASCRIPT

Copy

```
  'GET /users': (req, res, url) =>
    sendJson(res, 200, [...users.values()]),
};

http.createServer((req, res) => {
  const url = new URL(req.url, `http://${req.headers.host}`);
  const key = `${req.method} ${url.pathname}`;
  const handler = routes[key];
  if (handler) return handler(req, res, url);
  sendJson(res, 404, { error: 'not found' });
}).listen(3000);
```

Parsing query parameters, including repeated keys:

JAVASCRIPT Copy

```
// GET /search?q=node&tag=http&tag=tls
const url = new URL(req.url, `http://${req.headers.host}`);
const q = url.searchParams.get('q');         // 'node'
const tags = url.searchParams.getAll('tag'); // ['http', 'tls']
const all = Object.fromEntries(url.searchParams); // { q, tag: 'tls' } — l
```

Common mistakes & gotchas

- **Routing on `req.url` directly.** The query string makes `req.url === '/users'` false for `/users?x=1`. Route on `url.pathname`.
- **No default 404.** Forgetting the catch-all means unmatched requests never get a response and the client hangs.
- **Not distinguishing 404 from 405.** A known path with an unsupported method should be `405` with an `Allow` header, not `404`.
- **Falling through after responding.** Without `return` after `res.end()`, later branches may also write and throw `ERR_HTTP_HEADERS_SENT`.
- **Trailing-slash and case surprises.** `/users` and `/users/` are different strings; decide whether to normalise.

- **Trusting path segments as safe.** Using a path segment as a filename enables path-traversal (`../`); validate/normalise before touching the filesystem.
- **Assuming `Object.fromEntries(searchParams)` keeps duplicates.** It keeps only the last value; use `getAll()` for multi-value params.
- **Comparing against a percent-encoded pathname.** The `URL` class decodes `%2F` and friends in `pathname`, but a literal encoded slash inside a segment can still smuggle a `/` past a naive `split('/')`; validate decoded segments rather than trusting them as one token.
- **Awaiting the body on a route that has none.** Reading the stream on a `GET` just resolves empty, but forgetting to consume (or `req.resume()`) a body on a route that ignores it can, with some clients, hold the keep-alive socket until timeout.

Best practices

- **Always route on `url.pathname`**, parsed via the `URL` class.
- **Centralise response-sending** in a `sendJson` helper so headers and `end()` are consistent.
- **Return after responding** in every branch to prevent double-writes.
- **Emit `405` with `Allow`** for known paths hit with the wrong verb.
- **Set `Content-Length`** with `Buffer.byteLength(body)` for fixed JSON responses — it avoids chunked encoding overhead.
- **Validate input** before mutating state; reject with `400` and a clear message.
- **Keep a route table** once you have more than a handful of routes; it scales better than an `if/else` ladder.
- **Reach for a framework** (Express/Fastify) once you need middleware, param matching, and nested routers — hand-rolling is for learning and tiny services.

⭐ Interview lens

> **Q: Implement a minimal router without a framework.**
> Parse `req.url` with `new URL(req.url, base)`, then branch on `req.method` and `url.pathname`. Each branch responds and `return`s. End with a default `404`.

Show a dynamic route via regex/segment split for `/users/:id`, and a `405` for a known path with the wrong method.

Q: Why route on `pathname` and not `req.url`?
Because `req.url` includes the query string, so `req.url === '/users'` fails for `/users?active=1`. `url.pathname` strips the query, giving a stable key to match on.

Q: What's the difference between a 404 and a 405?
`404` means the resource/route does not exist at all. `405` means the path exists but the HTTP method is not supported for it — and the response must include an `Allow` header listing the valid methods.

Q: How do you extract a path parameter like the `42` in `/users/42`?
Either split `pathname` on `/` and read the segment, or match a regex with a capture group such as `/^\/users\/(\d+)$/` and take `match[1]`. Real routers compile `:param` patterns into exactly this kind of regex.

Q: Trap — you respond in one branch but the request still errors with "headers already sent." Why?
You forgot to `return` after `res.end()`, so control fell through to a later branch that wrote to `res` again. Each responding branch must `return` (or the logic must be mutually exclusive).

Q: How do you read a query parameter that can appear multiple times, like `?tag=a&tag=b`?
Use `url.searchParams.getAll('tag')`, which returns `['a','b']`. `get('tag')` returns only the first, and `Object.fromEntries` keeps only the last — both lose data for repeated keys.

- Routing = match `req.method` + `url.pathname` → handler.
- `const url = new URL(req.url, \`http://${req.headers.host}\`)`.
- Static: `pathname === '/users'`. Dynamic: regex `/^\/users\/(\d+)$/`.
- Query: `searchParams.get` / `.getAll` / `Object.fromEntries`.
- JSON: `Content-Type: application/json` + `JSON.stringify`.
- `return` after every response; end with a default `404`.
- Known path, wrong verb → `405` + `Allow` header.
- Scale up: route table, then a real framework.

QUICK CHECK — no marks, just for you.

1. Why should you route on url.pathname rather than req.url directly?

- ◯ req.url is always undefined until the body has been read
- ◯ req.url includes the query string, so req.url === "/users" fails for /users?active=1; pathname strips the query, giving a stable key
- ◯ req.url is lowercased by Node, breaking case-sensitive routes
- ◯ req.url is an array of segments, not a string, so it cannot be compared

Check answer

2. What is the difference between a 404 and a 405 response in a manual router?

- ◯ 404 is for server errors; 405 is for client errors
- ◯ 404 means the body was malformed; 405 means the headers were malformed
- ◯ 404 means the route does not exist at all; 405 means the path exists but the HTTP method is not supported for it (and must include an Allow header)

They are interchangeable; either is acceptable for an unmatched route

Check answer

3. You respond in one branch but still get "headers already sent". What is the most likely cause?

- ○ You set Content-Type twice, which double-sends the headers
- ○ You used sendJson instead of res.end, which sends headers implicitly
- ○ The client sent the request twice on the same socket
- ○ You forgot to return after res.end(), so control fell through to a later branch that wrote to res again

Check answer

‹ Previous
Parsing requests & writing responses

Next ›
HTTP vs HTTPS, TLS, HTTP/2 & CORS

LEARN MORE

ARTICLES & DOCS

- HTTP | Node.js Documentation
  nodejs.org
- Introduction to Node.js | Node.js Learn
  nodejs.org

VIDEOS



How to Handle Different Routes in Node.js HTTP Module | Build Custom Routing Logic

ProgrammingKnowledge

Hand-picked references for this lesson.